Université Mundiapolis
Campus Nouaceur
Filière : Ingénierie Informatique

TP 01

Module : Blockchain

Hachage SHA-256, chaînage de blocs et minage

Objectifs du TP

- Écrire une fonction qui calcule une empreinte **SHA-256** avec la bibliothèque `hashlib`.
- Représenter un bloc par un **dictionnaire** et relier les blocs par leur empreinte pour former une chaîne.
- Programmer le **minage** par preuve de travail (*Proof of Work*).
- Vérifier l'**intégrité de la chaîne** et mesurer le coût d'une falsification.

Prérequis

- Python : variables, chaînes, listes, dictionnaires, boucles `for` et `while`, fonctions.

1 Hachage SHA-256

Une fonction de hachage cryptographique transforme un texte de taille quelconque en une empreinte de taille fixe. SHA-256 produit 256 bits, soit 64 caractères hexadécimaux. Le même texte donne toujours la même empreinte, et la moindre modification du texte change presque toute l'empreinte (**effet avalanche**).

1.1 Importer `hashlib`. Écrire la fonction `calculer_sha256(texte)` qui renvoie l'empreinte SHA-256 de `texte`, puis afficher l'empreinte de `"Blockchain"`.
Utiliser `hashlib.sha256(texte.encode("utf-8")).hexdigest()`.

```
def calculer_sha256(texte):
    # à compléter
    return empreinte
```

```
625da44e4eaf58d61cf048d168aa6f5e492dea166d8bb54ec06c30de07db57e1
```

1.2 Ranger deux fois l'empreinte de `"Blockchain"` dans `h1` et `h2`, les afficher, puis afficher le résultat de `h1 == h2`.

```
625da44e4eaf58d61cf048d168aa6f5e492dea166d8bb54ec06c30de07db57e1
625da44e4eaf58d61cf048d168aa6f5e492dea166d8bb54ec06c30de07db57e1
Identiques ? True
```

1.3 Afficher, avec `len`, la longueur de l'empreinte de `"a"`, de `"Mundiapolis"` et de `"x" * 1000`.

```
1 caractere     -> 64
11 caracteres   -> 64
1000 caracteres -> 64
```

1.4 Écrire la fonction `compter_differences(h1, h2)` qui renvoie le nombre de positions `i` pour lesquelles `h1[i]` est différent de `h2[i]`. Afficher les empreintes de `"Bloc numero 1"` et `"Bloc numero 2"`, puis leur nombre de différences.
Utiliser un compteur initialisé à 0 et une boucle `for i in range(len(h1))`.

```python
def compter_differences(h1, h2):
    nb_differences = 0
    # à compléter
    return nb_differences
```

```
c17a909c42cb7d2acfb192bb9f75207a83cac136d2a006104a62b482b8b58793
587fa627c8742464522772db52e48634c1f40b5a4b7fc7e4a006c5f831059133
59 caracteres differents sur 64
```

1.5 Refaire l'étape 1.4 avec les transactions `"Ali envoie 12$ a Sara"` et `"Ali envoie 13$ a Sara"`.

```
572704069428e74fa353c4dd4a370f99f0af2ce99e6d755d2745e2eeab0aa538
02c23a118142da8378290b814975c081671720986bda11b7689d1c8d7179ecc1
62 caracteres differents sur 64
```

2 Représenter un bloc et chaîner

Un bloc est un **dictionnaire** à cinq clés. La liste `chaine` contient les blocs dans l'ordre. Le **chaînage** vient de ce que le hash d'un bloc est calculé avec le hash de son prédécesseur.

Clé	Contenu
`index`	position du bloc dans la chaîne : 0, 1, 2, ...
`donnees`	la transaction enregistrée dans le bloc
`nonce`	compteur utilisé par le minage (0 jusqu'à la partie 3)
`hash_precedent`	hash du bloc précédent (64 zéros pour le premier bloc)
`hash`	hash du bloc lui-même

Le registre des transactions utilisé dans tout le TP

Bloc	Transaction	Émetteur	Montant	Bénéficiaire
0	Bloc de genese Mundiapolis	aucune transaction		
1	Ali envoie 12$ a Sara	Ali	12$	Sara
2	Sara envoie 5$ a Karim	Sara	5$	Karim
3	Karim envoie 3$ a Nadia	Karim	3$	Nadia
4	Nadia envoie 7$ a Ali	Nadia	7$	Ali

Les transactions sont écrites sans accents afin que le texte haché soit identique sur toutes les machines. Recopier cette liste au début du programme :

```python
donnees_blocs = [
    "Bloc de genese Mundiapolis",
    "Ali envoie 12$ a Sara",
    "Sara envoie 5$ a Karim",
    "Karim envoie 3$ a Nadia",
    "Nadia envoie 7$ a Ali",
]
```

2.1 Écrire la fonction **`creer_bloc(index, donnees, hash_precedent)`** qui renvoie un dictionnaire contenant les cinq clés du tableau. Le `nonce` vaut `0` et le `hash` vaut `""` (chaîne vide). Afficher `creer_bloc(1, "Ali envoie 12$ a Sara", "abc")`.

```python
def creer_bloc(index, donnees, hash_precedent):
    bloc = ...    # dictionnaire à cinq clés
    return bloc
```

```
{'index': 1, 'donnees': 'Ali envoie 12$ a Sara', 'nonce': 0, 'hash_precedent': 'abc', '
    hash': ''}
```

2.2 Écrire la fonction **`calculer_hash_bloc(bloc)`** qui met bout à bout, dans cet ordre, l'index, le nonce, les données et le hash précédent du bloc, puis renvoie le `calculer_sha256` de ce texte.
`str(bloc["index"])` convertit l'entier en chaîne ; l'opérateur `+` concatène deux chaînes.

```python
def calculer_hash_bloc(bloc):
    contenu = ...    # index + nonce + donnees + hash_precedent
    return calculer_sha256(contenu)
```

2.3 Créer le bloc de genèse avec `creer_bloc(0, "Bloc de genese Mundiapolis", "0" * 64)`, calculer son hash et le ranger dans `genese["hash"]`. Créer ensuite la liste `chaine = [genese]` et afficher le hash de la genèse.

```
Hash de la genese : fb75966eb8d3dda427fec6a298feed4a9d5b0973343ba8a697e710a04f101599
```

2.4 Écrire la fonction **`ajouter_bloc(chaine, donnees)`** qui :

- récupère le dernier bloc, `chaine[len(chaine) - 1]`
- crée un bloc d'index `len(chaine)` dont le hash précédent est le hash du dernier bloc
- calcule le hash du nouveau bloc et l'ajoute à la liste avec `append`.

```python
def ajouter_bloc(chaine, donnees):
    # à compléter
    chaine.append(bloc)    # pas de return : la liste est modifiée
```

2.5 Ajouter les quatre transactions avec la boucle
`for i in range(1, len(donnees_blocs))`. Afficher ensuite, pour chaque bloc, son index et les dix premiers caractères de son hash précédent et de son hash (`[0:10]`).

```
Bloc 0 | prec : 0000000000 | hash : fb75966eb8
Bloc 1 | prec : fb75966eb8 | hash : 7f6af0a93a
Bloc 2 | prec : 7f6af0a93a | hash : f9adf24efc
Bloc 3 | prec : f9adf24efc | hash : 643f7518cb
Bloc 4 | prec : 643f7518cb | hash : 34b442a5b6
```

2.6 Écrire la fonction **`afficher_bloc(bloc)`** qui affiche chaque champ du bloc sur une ligne, puis afficher `chaine[1]`.

```python
def afficher_bloc(bloc):
    print("index          :", bloc["index"])
    # à compléter : une ligne par champ (pas de return)
```

```
index          : 1
donnees        : Ali envoie 12$ a Sara
nonce          : 0
hash_precedent : fb75966eb8d3dda427fec6a298feed4a9d5b0973343ba8a697e710a04f101599
hash           : 7f6af0a93ab7d810af731ff70ddc8c093d133a826c2bce7a2e73212c393ebff4
```

2.7 Écrire la fonction `lire_transaction(donnees)` qui renvoie l'émetteur, le montant (entier) et le bénéficiaire d'une transaction.
`donnees.split(" ")` donne `['Ali', 'envoie', '12$', 'a', 'Sara']` ; le montant s'obtient avec `int(mots[2].replace("$", ""))`.

```python
def lire_transaction(donnees):
    mots = donnees.split(" ")
    # à compléter
    return emetteur, montant, beneficiaire
```

```
('Ali', 12, 'Sara')
```

2.8 Écrire la fonction `calculer_soldes(chaine)` qui part du dictionnaire `{"Ali": 20, "Sara": 20, "Karim": 20, "Nadia": 20}`, parcourt les blocs à partir de l'index 1 (la genèse ne contient pas de transaction), retire le montant à l'émetteur et l'ajoute au bénéficiaire. Afficher le résultat.

```python
def calculer_soldes(chaine):
    soldes = {"Ali": 20, "Sara": 20, "Karim": 20, "Nadia": 20}
    # à compléter
    return soldes
```

```
{'Ali': 15, 'Sara': 27, 'Karim': 22, 'Nadia': 16}
```

3 Minage : preuve de travail

La preuve de travail impose que le hash du bloc commence par un nombre fixé de zéros, la **difficulté**. On essaie les valeurs du **nonce** 0, 1, 2, … jusqu'à obtenir un hash valide. Pour une difficulté de k zéros, il faut en moyenne 16^k essais, soit $16^4 = 65\,536$ essais pour $k = 4$.

3.1 Écrire la fonction `miner_bloc(bloc, difficulte)` qui :

- construit la cible `"0" * difficulte`
- met le nonce à 0 et calcule le hash du bloc
- tant que le hash ne commence pas par la cible, augmente le nonce de 1 et recalcule le hash
- compte les essais dans une variable `essais` (1 au départ, plus 1 à chaque tour) et la renvoie.

`not bloc["hash"].startswith(cible)` est vrai tant que le hash ne commence pas par la cible.

```python
def miner_bloc(bloc, difficulte):
    cible = "0" * difficulte
    # à compléter
    return essais
```

3.2 Créer un nouveau bloc de genèse, le miner avec la difficulté 4, puis afficher son nonce, le nombre d'essais et son hash.

```
Nonce trouve : 41489
Essais       : 41490
Hash         : 0000eb2c58b542efffaf1dcdada2baf8832a9bb933949c41442410ec8a9e7889
```

3.3 Écrire la fonction `construire_chaine(liste_donnees, difficulte)` qui part d'une liste vide et d'un hash précédent égal à `"0" * 64`, puis, pour chaque transaction :

- crée le bloc et le mine
- affiche une ligne `[mine] Bloc i : ... essais -> ...` avec les 20 premiers caractères du hash
- ajoute le bloc à la liste et retient son hash pour le bloc suivant.

La fonction renvoie la liste des blocs.

```python
def construire_chaine(liste_donnees, difficulte):
    chaine = []
    hash_precedent = "0" * 64
    # à compléter
    return chaine
```

3.4 Miner les cinq blocs du registre : `chaine = construire_chaine(donnees_blocs, 4)`.

```
[mine] Bloc 0 : 41490 essais -> 0000eb2c58b542efffaf...
[mine] Bloc 1 : 48145 essais -> 0000fb41e6cae14ae3b7...
[mine] Bloc 2 : 22968 essais -> 00008ddc86166eaece69...
[mine] Bloc 3 : 13919 essais -> 0000cc4bf72544ba469c...
[mine] Bloc 4 : 20150 essais -> 00009538d1e384c589b8...
```

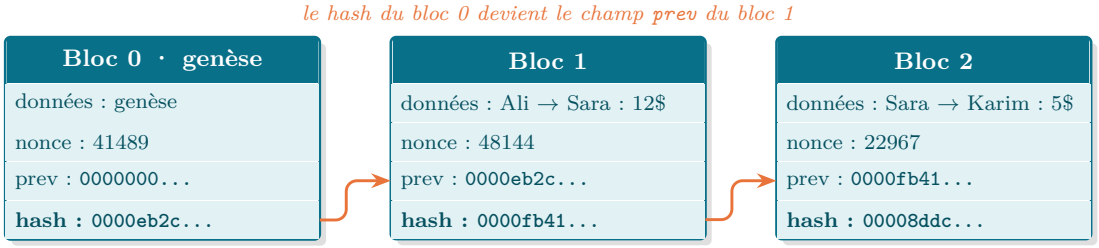


FIGURE 1 – Les trois premiers blocs minés à la difficulté 4 : le champ `prev` recopie le hash du bloc précédent.

4 Visualiser et vérifier la chaîne

4.1 Écrire la fonction **`afficher_chaine(chaine)`** qui dessine chaque bloc dans un cadre de tirets (hash tronqués à 28 caractères) et place une flèche verticale entre deux blocs, sauf après le dernier. Appeler `afficher_chaine(chaine)`.

```
def afficher_chaine(chaine):
    for i in range(len(chaine)):
        bloc = chaine[i]
        # à compléter : cadre du bloc, puis flèche si i < len(chaine) - 1
```

```
+------------------------------------------------+
| BLOC 0
+------------------------------------------------+
| Donnees      : Bloc de genese Mundiapolis
| Nonce        : 41489
| Hash prec.   : 00000000000000000000000000...
| Hash du bloc : 0000eb2c58b542efffaf1dcdada2...
+------------------------------------------------+
                        |
                        v
          (blocs 1, 2, 3 et 4 sur le meme modele)
```

4.2 Écrire la fonction **`verifier_chaine(chaine, difficulte)`** qui effectue, pour chaque bloc `i`, les trois contrôles suivants dans cet ordre. Au premier contrôle qui échoue, elle affiche le message indiqué et renvoie `False` ; si tous les contrôles réussissent, elle renvoie `True`.

Contrôle	Message en cas d'échec
`calculer_hash_bloc(bloc)` égal à `bloc["hash"]`	`Bloc i : le hash recalcule ne correspond pas au hash stocke`
`bloc["hash"]` commence par la cible	`Bloc i : la preuve de travail n'est pas satisfaite`
pour `i > 0`, `bloc["hash_precedent"]` égal à `chaine[i - 1]["hash"]`	`Bloc i : hash_precedent different du hash du bloc i-1`

```
def verifier_chaine(chaine, difficulte):
    cible = "0" * difficulte
    for i in range(len(chaine)):
        # à compléter : les trois contrôles, return False au premier échec
    return True
```

4.3 Afficher `verifier_chaine(chaine, 4)`.

```
Chaine valide ? True
```

5 Augmenter la difficulté

Passer de la cible `0000` à la cible `00000` ajoute un zéro : le nombre moyen d'essais passe de $16^4 = 65\,536$ à $16^5 = 1\,048\,576$ par bloc.

5.1 Écrire la fonction **`compter_essais(chaine)`** qui renvoie le nombre total d'essais de la chaîne. Un bloc dont le nonce vaut n a demandé $n + 1$ essais.

```
def compter_essais(chaine):
    total = 0
    # à compléter
    return total
```

5.2 Importer `time`. Mesurer la durée de `construire_chaine(donnees_blocs, 4)` : noter `debut = time.time()` avant l'appel, puis `duree4 = time.time() - debut` après. Ranger la chaîne dans `chaine4`, le total des essais dans `total4`, et afficher ce total avec la durée arrondie (`round(duree4, 2)`).

```
[mine] ...    (les 5 lignes de l'etape 3.4)
Total : 146672 essais en 0.27 s
```

5.3 Refaire la même mesure avec la difficulté 5 (`chaine5`, `duree5`, `total5`).

```
[mine] Bloc 0 : 985225 essais -> 00000527236c51741ed5...
[mine] Bloc 1 : 272033 essais -> 00000af5c35071593b42...
[mine] Bloc 2 : 169506 essais -> 000004ba3e46a356355c...
[mine] Bloc 3 : 446240 essais -> 00000edcbadc3c1dda49...
[mine] Bloc 4 : 355141 essais -> 00000ae0d4a0deac94f5...
Total : 2228145 essais en 5.37 s
```

5.4 Afficher le rapport `total5 / total4` arrondi à une décimale, puis le débit de la machine, `debit = total5 / duree5`, en hash par seconde. Calculer enfin le temps nécessaire pour une cible de 8 zéros, `16 ** 8 / debit`, en secondes puis en heures.

```
Rapport des essais : 15.2
Debit : 414858 hash par seconde
Estimation pour 8 zeros : 10353 s, soit 2.9 heures
```

5.5 Remplir le tableau ci-dessous avec les nombres d'essais affichés aux étapes 5.2 et 5.3 et avec vos durées.

Bloc (transaction)	Essais, cible `0000`	Essais, cible `00000`	Rapport
0 : genèse	…	…	…
1 : Ali → Sara 12$	…	…	…
2 : Sara → Karim 5$	…	…	…
3 : Karim → Nadia 3$	…	…	…
4 : Nadia → Ali 7$	…	…	…
Total	**…**	**…**	**…**
Durée	**…s**	**…s**	**…**

6 Falsifier un montant

La chaîne minée à la difficulté 4 est valide. Ali veut faire croire qu'il a envoyé un montant bien supérieur à Sara : il modifie le bloc 1 sans toucher au reste de la chaîne.

`"Ali envoie 12$ a Sara"` ⟶ `"Ali envoie 2500$ a Sara"`

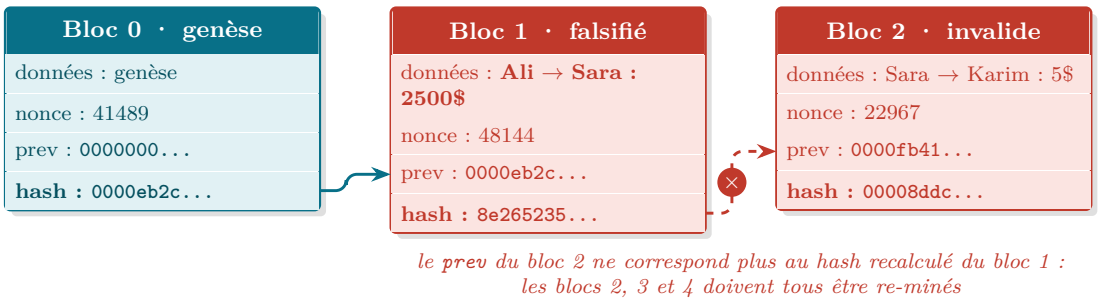


FIGURE 2 – La modification du montant du bloc 1 change son hash et rompt le chaînage de tous les blocs suivants.

6.1 Reconstruire `chaine` à la difficulté 4 et afficher `verifier_chaine(chaine, 4)`. Remplacer ensuite `chaine[1]["donnees"]` par `"Ali envoie 2500$ a Sara"`, sans rien recalculer, et vérifier à nouveau.

```
[mine] ...    (les 5 lignes de l'etape 3.4)
Chaine valide ? True
Bloc 1 : le hash recalcule ne correspond pas au hash stocke
Chaine valide ? False
```

6.2 Afficher le hash stocké du bloc 1, puis son hash recalculé avec `calculer_hash_bloc`.

```
hash stocke    : 0000fb41e6cae14ae3b79dfec78a52e6504a09d7e3cbe8aba28b9263eaa215a9
hash recalcule : 8e265235c949f39c7809bf18bc9df3d44e8fe463def4471654281f38e1e57e6d
```

6.3 Re-miner seulement le bloc 1 avec `miner_bloc(chaine[1], 4)`, afficher le nombre d'essais, puis vérifier la chaîne.

```
Bloc 1 re-mine en 31895 essais -> 0000f5c9f0d67d186e41...
Bloc 2 : hash_precedent different du hash du bloc 1
Chaine valide ? False
```

6.4 Écrire la fonction **`reminer_a_partir_de(chaine, debut, difficulte)`** qui, pour chaque index `i` de `debut` jusqu'au dernier bloc :

- recopie le hash du bloc `i - 1` dans le `hash_precedent` du bloc `i` (si `i > 0`)
- mine le bloc `i` et affiche une ligne `[re-mine] Bloc i : ... essais -> ...`
- ajoute ses essais à un total.

La fonction renvoie le total des essais.

```python
def reminer_a_partir_de(chaine, debut, difficulte):
    total = 0
    for i in range(debut, len(chaine)):
        # à compléter
    return total
```

6.5 Reconstruire la chaîne, falsifier le bloc 1, puis appeler `cout_bloc1 = reminer_a_partir_de(chaine, 1, 4)`. Afficher le total et vérifier la chaîne.

```
[mine] ...    (les 5 lignes de l'etape 3.4)
[re-mine] Bloc 1 : 31895 essais -> 0000f5c9f0d67d186e41...
[re-mine] Bloc 2 : 32567 essais -> 00003952799ef40f2394...
[re-mine] Bloc 3 : 5397 essais -> 00004ff8de4bc444ef57...
[re-mine] Bloc 4 : 78354 essais -> 000042c383bce62f9ca8...
Total : 148213 essais
Chaine valide ? True
```

6.6 Reconstruire la chaîne et falsifier cette fois le dernier bloc : `chaine[4]["donnees"] = "Nadia envoie 700$ a Ali"`.
Re-miner avec `cout_bloc4 = reminer_a_partir_de(chaine, 4, 4)`, puis afficher les deux coûts.

```
[mine] ...    (les 5 lignes de l'etape 3.4)
[re-mine] Bloc 4 : 29829 essais -> 0000086e30076da93b8d...
Falsifier le bloc 1 : 148213 essais
Falsifier le bloc 4 : 29829 essais
```

6.7 Refaire l'étape 6.5 avec la difficulté 5 (construction et re-minage), ranger le total dans `cout_bloc1_d5`, puis afficher le facteur `cout_bloc1_d5 / cout_bloc1` arrondi à une décimale.

```
[mine] ...    (les 5 lignes de l'etape 5.3)
[re-mine] Bloc 1 : 855060 essais -> 00000413c0e7f1d47eeb...
[re-mine] Bloc 2 : 1202634 essais -> 00000336e28689cda4ba...
[re-mine] Bloc 3 : 176401 essais -> 000001e7cdf2a731fc76...
[re-mine] Bloc 4 : 198623 essais -> 00000a619dace353cf4f...
Difficulte 4 : 148213 essais
Difficulte 5 : 2432718 essais
Facteur      : 16.4
```

Changer un seul caractère du montant fait perdre au hash ses zéros de tête. Après re-minage, le bloc 1 possède un nouveau hash que le bloc 2 ne référence pas : il faut re-miner le bloc 2, puis le bloc 3, et ainsi de suite. Falsifier une transaction ancienne coûte le re-minage de toute la fin de la chaîne (148 213 essais ici), alors que falsifier la dernière ne coûte qu'un bloc (29 829 essais). Avec un zéro de plus, ce coût est multiplié par 16 environ.